

UNIDAD 2: MANIPULACIÓN DE DATOS

1

Interpolación lineal: toma dos puntos y traza una recta para conocer el punto medio entre ellos.

Interpolación polinómica: en lugar de una recta busca un polinomio que pase por todos los puntos conocidos.

```
import pandas as pd
```

```
dataframe = pd.read_csv('database.csv')
```

```
dataframe.info()
```

```
# pd.read_csv('database', dtype = {'price': 'float'})
```

Cambiar tipo de dato: `df['Age'] = df['Age'].astype('int64')`

Arreglo solo con valores únicos: `dataframe['nombre'].unique()`

Valores únicos y cantidad de repeticiones: `dataframe['name'].value_counts()`

Máximos y mínimos: `dataframe['edad'].max()` devuelve V o F.

MERGE: `pd.merge(dataframe1, dataframe2, on = 'columna', how = 'left/right/inner')`

Expresiones regulares:

- `ld` busca un dígito
 - `+` una o más ocurrencias
- } `ld+` captura una secuencia de uno o más caracteres.

"Total a pagar: USD 45000".

`dataframe[['Moneda', 'Monto']] = dataframe['columna_texto'].str.extract(r'([A-Za-z]+)\s(ld+)')`

⇒

Moneda	Monto
USD	45000

UNIDAD 3: ANÁLISIS EXPLORATORIO DE DATOS, MEDIDAS DE RESUMEN

- Media aritmética: promedio clásico
 - Mediana: valor del medio con los datos ordenados de - a +
 - Moda: valor que más se repite
- } medidas de posición (centralidad)
- Rango: diferencia entre valor más chico y más grande
 - Varianza: (σ^2) mide cuánto se desvían en promedio todos los datos respecto a la media aritmética
 - Desvío estándar: (σ) raíz cuadrada positiva de la varianza.
- } medidas de dispersión

- Coeficiente de variación (CV): $CV = \frac{s}{\bar{x}}$. $\bar{x}$ = medio. Útil para comparar la variabilidad entre cosas muy distintas.

A prueba de valores atípicos:

- Rango intercuartil: (RI) diferencia entre el primer y tercer cuartil. Vemos cómo se dispersa el 50% del "medio" de nuestros datos ignorando los extremos.
- Desviación mediana absoluta: (MAD) calcula las diferencias absolutas entre cada dato y la mediana y luego se toma la mediana a esas dif.

[cuando usamos la mediana como medida de centralidad, la medida de dispersión que se considera adecuada para acompañarlo es el RI].

Covarianza: evalúa cómo varían dos variables de forma conjunta. (S_{xy}).

$S_{xy} > 0 \rightarrow$ relación directa (son proporcionales).

$S_{xy} < 0 \rightarrow$ relación inversa (inversamente proporcionales).

$S_{xy} = 0 \rightarrow$ sin relación lineal.

Coeficiente de correlación de Pearson: versión estandarizada de la S_{xy} .

$-1 \leq r \leq 1$. A más cercano al 1, mayor relación entre las variables.

Ojo!! sólo mide relaciones lineales y no tolera bien los outliers.

Coeficiente de correlación de Spearman: (r_s). es menos sensible a los outliers y detecta relaciones monótonas.

COMANDOS:

`df['ingresos'].mean()` $\rightarrow$ medio.

`df['ingresos'].median()` $\rightarrow$ mediana. `np.percentile(df['ingresos'], q=50)` con Numpy.

`df['ingresos'].mode()` $\rightarrow$ moda.

`df['ingresos'].max() - df['ingresos'].min()` $\rightarrow$ rango.

`df['ingresos'].var()` $\rightarrow$ varianza

`df['ingresos'].std()` $\rightarrow$ desviación estándar

`q1 = df['ingresos'].quantile(q=0.1)` $\rightarrow$ cuartiles. $q_1 = 0.25$, $q_3 = 0.75$

`df['ingresos'].describe()` $\rightarrow$ devuelve cantidad de datos, medio, dev. estándar, mínimo, cuartiles y máximo.

`sns.boxplot(x='ingresos', data=df)` muestra outliers. para ocultarlos agregar `showliers=False`.

`sns.scatterplot(x='edad', y='ingreso', data=df)` gráfico de dispersión

`df[['edad', 'ingresos']].cov()` matriz de covarianza.

`df[['edad', 'ingresos']].corr(method='spearman')` matriz de correlación.

`matriz = df[['edad', 'ingresos']].corr()` } gráfico de calor (corulograma).
`sns.heatmap(matriz, annot=True)` (*)

UNIDAD 4: ANÁLISIS EXPLORATORIO DE DATOS, VISUALIZACIONES

serie temporal: sucesión de observaciones ordenadas en el tiempo, las mismas no son independientes.

Gráfico de líneas:

agrega un punto en cada observación real.

`sns.lineplot(x='anio', y='porcentaje', data=df, marker='o')`

para ver varios valores al mismo tiempo: `hue='pais'`.

Para evitar la superposición de líneas usamos el heatmap

Pairplot → genera todos los scatterplots posibles entre las variables que se indiquen, organizados en una matriz.

`sns.pairplot(df, vars=['var1', 'var2', 'var3', 'var4'])`.

- fuera de la diagonal → un scatterplot por cada par de variables.
- en la diagonal: un histograma de cada variable individual.

jagando `hue='var'` cada variación de la misma se coloca distinto y la diagonal pasa de histogramas a curvas de densidad por especie, permitiendo ver si las relaciones entre variables cambian según el grupo.

Corulograma: es un heatmap construido sobre la matriz de correlación. El coeficiente de correlación r resume en un sólo número qué tan fuerte y en qué dirección es la relación lineal entre dos variables. (*)

podemos agregar... `cmap='BrBG', vmin=-1, vmax=1`) para que la escala de colores esté centrada en 0.

Para incorporar variables cuantitativas:

Bubble chart: scatterplot donde el tamaño de cada punto representa una tercera variable cuantitativa.

agregamos a `sns.scatterplot`: `size = 'var. cuantitativa'`

gráfico de barras: valor chico para barras finas

`sns.countplot` (`x = 'var'`, `stat = 'percent'`, `width = 0.2`, `data = df`).

Histograma de frecuencias → representar datos de una variable cuantitativa que asume un gran no. de valores diferentes
 abscisas → intervalos en que se agrupan los valores de la variable.
 ordenadas → frecuencia absoluta o relativa.

`sns.histplot` (`x = 'age'`, `data = df`) podemos cambiar los bins (cantidad de barras `bins = sqrt`, `bins = 10`, `bins = [0, 5, 10, 15]`, `binwidth` (amplitud de cada intervalo), `binrange` (rango cubierto por el histograma).

Gráfico de densidad: hue = 'specie' para superponer distintas variables (especie)
 → `multiple = 'layer'`

`sns.kdeplot` (`x = 'age'`, `fill = True`, `data = df`) . A veces la curva se extiende sobre datos que no existen en el df, agregar: `clip = [min, max]`

`plt.xlim(min, max)` → ajustar la vista sacando los huecos en blanco.

podemos modificar los picos con `bw = ...`

Agregar línea al gráfico de densidad al histograma con `kde = True`

Boxplot → me permite ver outliers.

Gráfico de densidad → permite ver máximos locales.

`sns.boxplot` (`x = 'var'`, `y = 'species'`, `hue = 'species'`, `data = df`)

Strip/jitter plot.

`sns.stripplot` (`x = 'var'`, `y = 'species'`, `hue = 'species'`, `jitter =`

muestra todos los datos, no los resume.

Gráfico de barras:

`sns.countplot` (`y = 'class'`, `stat = 'percent'`, `data = df`)

variables cualitativas → barras horizontales

variables cuantitativas → barras verticales

para representar al mismo tiempo más de una variable categórica a la vez: agrupamos hue: 'var2'.

Barra agrupadas o paralelas:

```
pd.crosstab(df['var1'], df['var2'], normalize = 'all').mul(100).round(2)
```

no devuelve una gráfica!! arma una tabla de contingencia

```
tabla = ...
```

```
tabla.plot.barh(stacked=True)
```

Distribución conjunta: para llegar al 100% hay que sumar los porcentajes de todas las barras.

Distribución condicional: los porcentajes suman 100% dentro de cada una de las clases.

Gráfico de sectores/pasta:

```
tabla = df['var'].value_counts(normalize = True).mul(100).round(2)
```

```
plt.pie(data = tabla, labels = 'Variable', x = 'Porcentaje', autopct = '%.1f%%')
```

```
plt.legend(title = '...', labels = ['...', '...', '...'])
```

Convertir a formato largo:

```
pd.melt(df, id_vars = 'País', var_name = 'Año', value_name = 'Porcentaje')
```

↓
columnas que sirven como indicadores, en las que la var. está relacionada ahí.

nombre de columna que va a contener los valores que apilaste

↓
nombre de columna que va a contener los valores que estaban en su dataframe

Encalas de colores divergentes → cmap = 'vlag', 'RdYlGn', 'BrBG', 'coolwarm', vmin = x1, vmax = x2

↳ para Heatmap!!

~~UNIDAD 5: TRANSFORMACIÓN DE DATOS: TÉCNICAS DE ESCALADO~~

~~Término Translacional: (T) mueve los datos a través del eje x.~~

~~Factor de escalado: (S) hace que los datos estén más dispersos o concentrados horizontalmente.~~

Fundamentos de la Ciencia de Datos

6

Similaridad de Jaro.

$$sim_j = \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right)$$

s_i = longitud de la cadena s_i

m = nro. de caracteres que coinciden *

t = mitad del nro. de caracteres transpuestos, de los que coinciden.

* coinciden si se encuentran
máximo a $\frac{\max(|s_1|, |s_2|)}{2} - 1$

valor más cercano a 1 $\rightarrow$ más similaridad.
más cercano a 0 $\rightarrow$ menos similaridad.

$\Rightarrow$ pip install jaro-winkler

jaro-winkler:

$$sim_{jw} = sim_j - lp(1 - sim_j)$$

l = nro. de caracteres que coinciden al principio de la cadena con un max. de 4.

p = factor escalado. estándar = 0,1

cant. de caract. a redondear

Cálculo en Python: $sim_j = \text{round}(\text{jaro.jaro_metric}('s_1', 's_2'), 4)$

$sim_{jw} = \text{round}(\text{jaro.jaro_winkler_metric}('s_1', 's_2'), 4)$

Distancia de edición de Levenshtein

cuenta la menor cantidad de operaciones que son necesarias para convertir una cadena a otra (inserción, borrado, sustitución). (+ alto + distintos)

Similaridad de Levenshtein

$$sim_{leven} = 1 - \frac{\text{dist}_{leven}(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad + \text{alto} + \text{similar}$$

Fuzzy joins $\rightarrow$ enlaces difusos, se usan para matchear datos que pueden no coincidir perfectamente debido a posibles inconsistencias, omisiones o errores. se buscan coincidencias aproximadas.

`fuzz.token_sort_ratio(name, row2['nombre'])` $\rightarrow$ tokeniza, asignamos un valor numérico de similitud (puntaje).

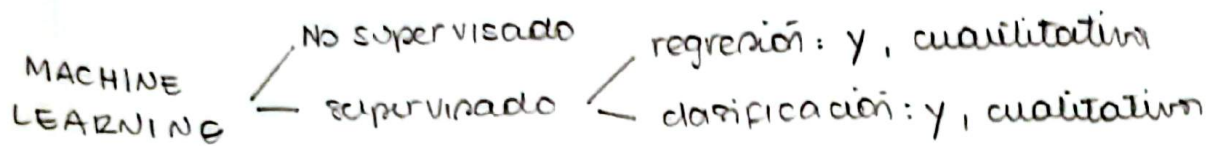
Distancias y medidas de similitud en datos numéricos

Clustering $\rightarrow$ agrupar elementos en grupos homogéneos (clusters) en función de la similitud entre ellos.

Detección de outliers $\rightarrow$ identificar elementos muy diferentes al resto.

clasificación de vecinos más cercanos (KNN): clasificar elementos

basándose en las características de otros elementos similares.



Distancia Euclídea

medida + popular. es sensible a las unidades de las variables, normalizar y estandarizar los datos. (mide la dist. entre 2 variables en un vector).

$$d_E(i, j) = \sqrt{(x_{1i} - x_{1j})^2 + (x_{2i} - x_{2j})^2 + \dots}$$

Scipy → calculos y representacion matrices de distancias.

- `pdist()`: calcula todas las distancias por pares entre filas de una matriz o dataframe.
- `squareform()`: convierte el resultado de `pdist` (vector) en una matriz cuadrada simétrica.

Distancia Manhattan: sólo mide distancias avanzando en líneas rectas verticales u horizontales. $d_{MHT}(i, j) = |x_{1i} - x_{1j}| + |x_{2i} - x_{2j}| + \dots$

Distancia de Mahalanobis: mide qué tan lejos está un punto respecto a cómo se distribuyen los datos, considerando la variabilidad / dispersión de los mismos, la estructura de correlación entre variables. se suele usar para la detección de valores atípicos. Podemos calcular la distancia al centroide.

ANÁLISIS DE REGRESIÓN: conjunto de herramientas que permite modelar el comportamiento de una variable respuesta Y (cuantitativas continuas) a partir de la información proporcionada por una o más variables predictoras $x_1, x_2, \dots, x_p$.

$$Y = f(x_1, x_2, \dots, x_p) + E \quad E = \text{error aleatorio, recoge todo lo no incluido en el modelo.}$$

función de regresión

Para variables con relación lineal: $f = \beta_0 + \beta_1 X + E$

β_0 = ordenada al origen. valor promedio de la variable respuesta cuando la explicativa es 0.

β_1 = pendiente. cambio promedio en la variable respuesta cuando la explicativa aumenta en una unidad.

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

$\hat{y}$ = y predicción. $\hat{\beta}_0$ y $\hat{\beta}_1$ = estimaciones puntuales de los parámetros poblacionales β_0 y β_1 .

Método de Mínimos Cuadrados: (OLS) provee las estimaciones que mejor ajusta a los datos experimentales.

Las distancias verticales de los puntos observados a la recta son los residuos de modelo. $e_i = y_i - \hat{y}_i = y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)$

```
smf.ols(formula = 'var.respuesta ~ var.predictora', data = df).fit() #ALT+126: ~  
= modelo  $\hat{\beta}_0$ : Intercept + coef  
print(modelo.summary())  $\hat{\beta}_1$ : var.predictora + coef
```

```
sns.lmplot(x = 'var.predictora', y = var.respuesta, ci = False, data = df)
```

Coefficiente de determinación: (R^2) mide la proporción de la variabilidad total en las observaciones de la variable respuesta y que es explicada por el modelo. $0 \leq R^2 \leq 1$. $0 \rightarrow$ modelo pésimo. $1 \rightarrow$ modelo óptimo

No garantiza 100% que un modelo sea bueno, puede ser una relación no lineal, estar sobreadjustado, valores atípicos influyentes, etc.

Regresión lineal múltiple $\rightarrow$ más de una variable predictora.

predecir la masa corporal de un pingüino de esta región en función de la longitud de su alar.

```
modelo = smf.ols(formula = 'body-mass.g ~ flipper-length.mm', data = df).fit()  
sns.lmplot(x = 'flipper-length.mm', y = 'body-mass.g', ci = False, data = df)
```

agregar sex como variable predictora: creación de variables dummy.

$$x_2 = \begin{cases} 1 & \text{si es macho} \\ 0 & \text{si es hembra} \end{cases} \quad y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon$$

Creación de variables **DUMMY**:

```
df['male-dummy'] = df['sex'].apply(lambda x: 1 if x == 'Male' else 0)
```

```
modelo = smf.ols(formula = 'body-mass ~ flipper + male-dummy', data = df).fit()
```


INFERENCIA

9

Ensayar hipótesis sobre los parámetros.

Hipótesis nula: H_0 , no hay relación entre X_1 e Y . $\beta_1 = 0$.

Hipótesis alternativa: H_1 , sí hay relación lineal entre X_1 e Y . $\beta_1 \neq 0$.

H_0 es siempre cierta hasta que se demuestre lo contrario.

Valor-p: probabilidad de obtener por puro azar y asumiendo que H_0 es verdadera una estimación tan alejada de 0 como la que observamos (o más). Un valor-p pequeño indica que sería muy improbable observar lo que observamos si el predictor no tuviera efecto real. Por lo tanto de evidencia contra H_0 .

P-value $< 0,05 \rightarrow$ rechazamos H_0 . + estricto 0,01, + flexible 0,1.

R^2 ajustado: penaliza la inclusión de variables que no mejoran genuinamente el ajuste. mayor R^2_{adj} $\checkmark$.

SIEMPRE ME QUEDO CON EL MODELO MÁS SIMPLE!!!

p value en Tabla.

$P > |t| \rightarrow$ valores mayores a 0,05 debemos descartarlos. 

ejemplo para predicción: inversión de 150 mil USD TV, 30 mil USD radio, 60 mil USD diario

`datos_prediccion = pd.DataFrame({'TV': [150], 'Radio': [30], 'Newspaper': [60]})`

`modelo.predict(datos_prediccion)`. (para no hacer las cuentas a mano).

Si hago un histograma de los errores E y son asimétricos $\rightarrow$ mal

Normal Q-Q Plot: si los puntos siguen aproximadamente la recta de referencia, el supuesto se cumple razonablemente.

Si la relación verdadera está lejos de ser normal, las conclusiones que saquemos del ajuste son poco confiables. Si hacemos un gráfico de residuos vs. los valores predichos y los puntos siguen un patrón sistemático, la relación no es lineal.

Heterocedasticidad: que los valores de error E no tienen varianzas o variabilidad constante. (presencia de patrones tipo muelle o embudo en el gráfico de residuos vs. valores predichos, no se dispersan uniformemente al rededor de 0). Se puede resolver aplicando alguna transformación como la respuesta Y usando una función cóncava ($\log(Y)$, $\sqrt{Y}$) que tienden a estabilizar la varianzas. Si prediceamos con el modelo cambiado obtenemos "drapificar" esas transformaciones a la respuesta.

Multicolinealidad: cuando dos o más predictores están fuertemente correlacio-

dados entre si. Analizamos la matriz de correlación de los predichos.

10

Gráfico de residuos vs. valores predichos:

```
sns.regplot(x = modulo.pittedvalues, y = modelo.resid, lowess = True)
```

```
plt.axhline(y = 0, color = 'black', linestyle = '- -')
```

calculamos los predichos: `datos['predichos'] = modulo.predict(datos)`
(`modulo.pittedvalues`).

residuo (y observado - y predicho): `datos['residuos'] = datos['y'] - datos['predicho']`
(`modulo.resid`)

Normal Q-Q plot de residuos: `sm.qqplot(modulo.resid, line = 's')`. usar para completar análisis cuando el gráfico de residuos vs. predichos da una distribución simétrica.

LIBRERÍAS

```
import pandas as pd
" numpy as np
" statsmodels.api as sm
" statsmodels.formula.api as smf
" matplotlib.pyplot as plt
" seaborn as sns
from scipy import stats
```